



Sistemas Operativos en Tiempo Real

Codiseño HW/SW – Sistemas Integrados Digitales

Marcos Martínez Peiró mpeiro@eln.upv.es

*Máster Universitario de Ingeniería de Sistemas Elctrónicos – MUISE –
Máster Universitario de Ingeniería de Telecomunicaciones – MUIT -*

2024/2025

Rtos –Real Time Operating Systems

Organización

- Introducción
 - Concepto de Sistema Embebido
 - Arquitectura SW de un Sistema Embebido
 - Concepto de Rtos
 - Principales Rtos
 - Embedded Linux vs Rtos
- Layered Software Architecture
 - Driver
 - HAL
 - MCAL – Middleware
 - Application
 - Rtos

Rtos –Real Time Operating Systems

Organización

- RTOS
 - Características de un RTOS
 - Tareas
 - Scheduler en RTOS
 - Comunicación entre tareas
 - Profiling en RTOS
- Micro C RTOS
- Sesiones de Laboratorio

Sistema Embebido

Concepto de Sistema Embebido

- Es un sistema digital con recursos limitados.
 - Memoria, capacidad de proceso, periféricos.
- Contiene un HW específicamente diseñado para una aplicación concreta.
 - Reducción de costes, optimización de recursos, periféricos específicos.
- Contiene un SW acorde con las limitaciones hardware.
 - Es suficientemente rápido para la aplicación para la que se diseñó (Real-Time).
 - Es compacto, ocupa la menor cantidad posible de memoria para abaratar el coste del producto.
- A menudo funcionan dentro de sistemas más grandes.
 - Son un equipo electrónico que debe conectarse o comunicarse con el resto de los equipos del sistema completo.
 - Realizan una función determinada en el producto complejo final.

Sistema Embebido

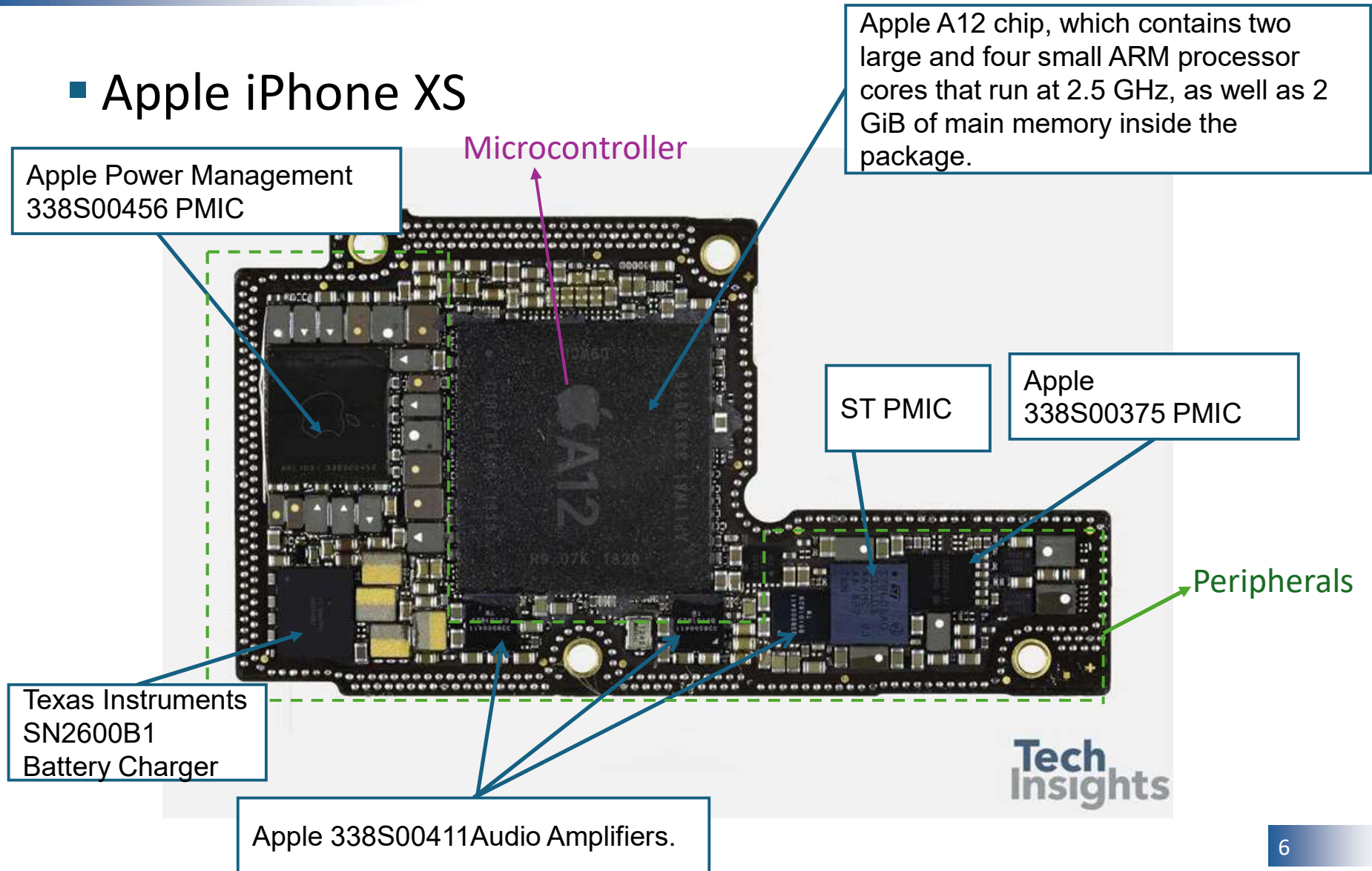
Concepto de Sistema Embebido

- Algunos ejemplos de sistemas embebidos:
 - Tarjetas electrónicas en un automóvil.
 - Tarjetas electrónicas en sistemas de aviación.
 - Tarjeta controladora en un ascensor.
 - Tarjeta controladora en un microondas.
 - Tarjeta de control en un inversor fotovoltaico.
 - Tarjeta electrónica en una lavadora.
 - Tarjetas en juguetes.
 - Tarjetas en telefonía móvil.
 - Tarjetas de sistemas multimedia.
- No son una CPU genérica sino dedicada para la aplicación concreta.

Sistema Embebido

Concepto de Sistema Embebido

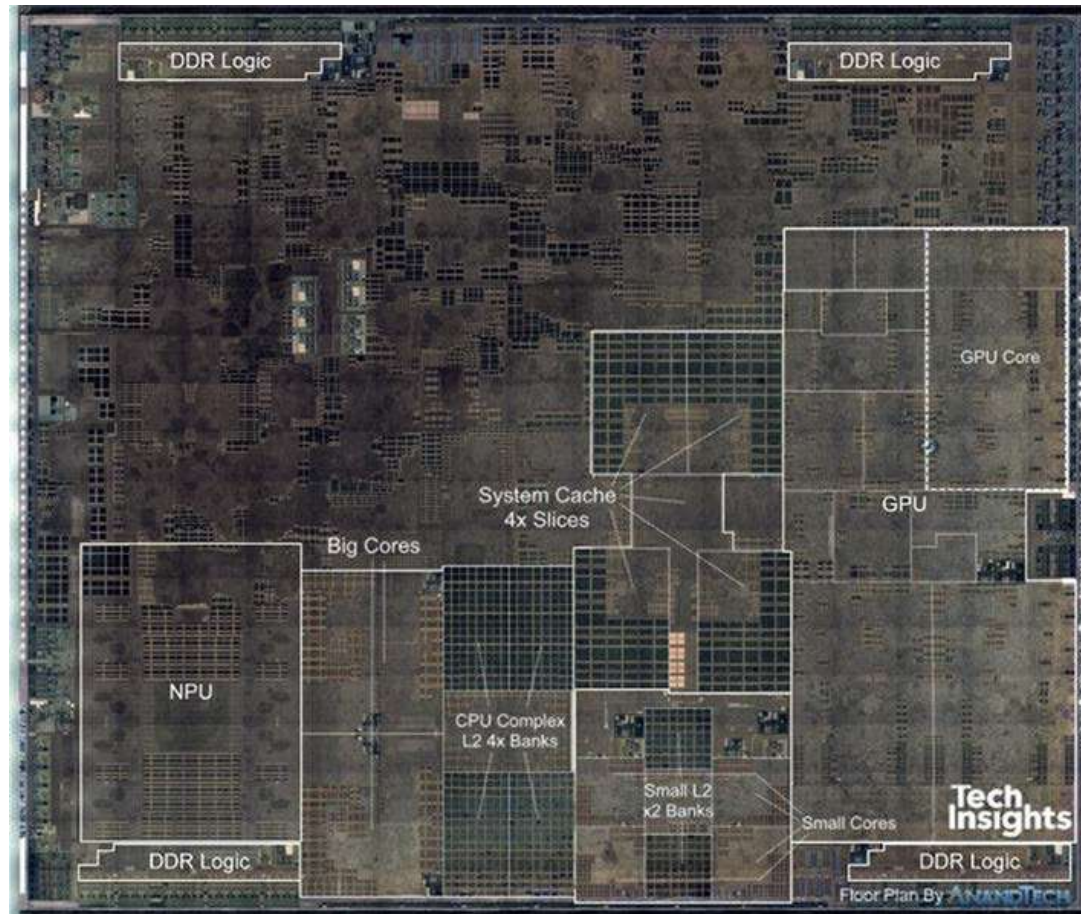
■ Apple iPhone XS



Sistema Embebido

Concepto de Sistema Embebido

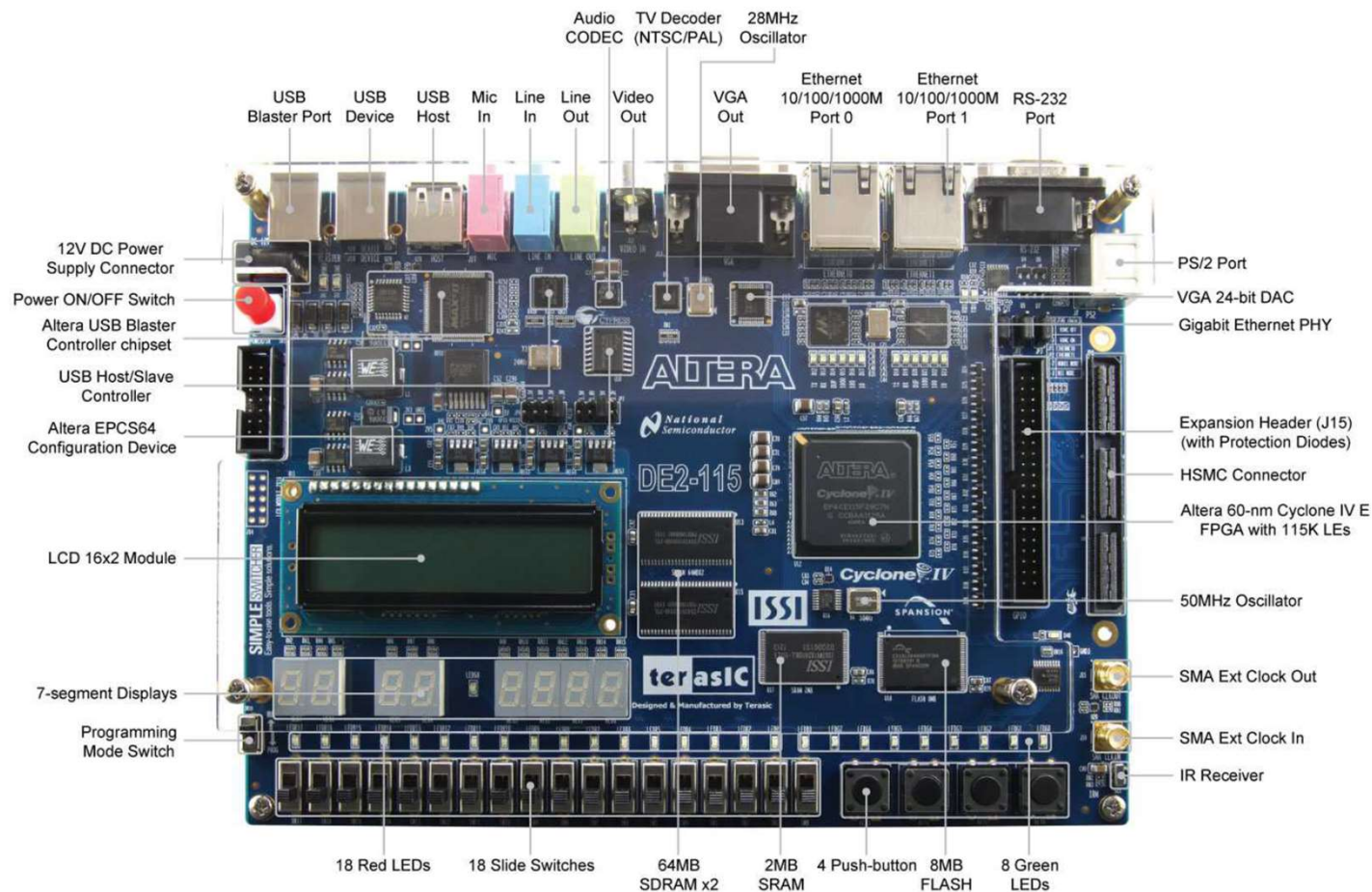
- Dentro del chip A12



Sistema Embebido

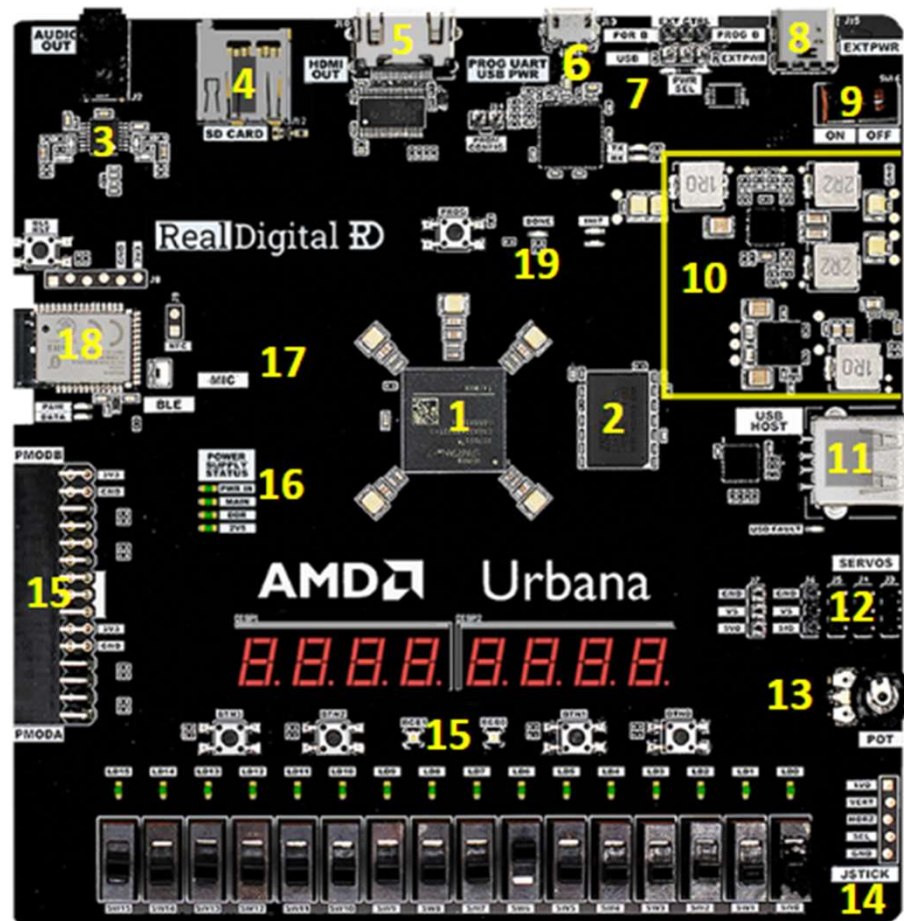
Embebido FPGA

■ Terasic DE2-115 FPGA Board



Embebido FPGA

- RealDigital Urbana Spartan7 FPGA Board

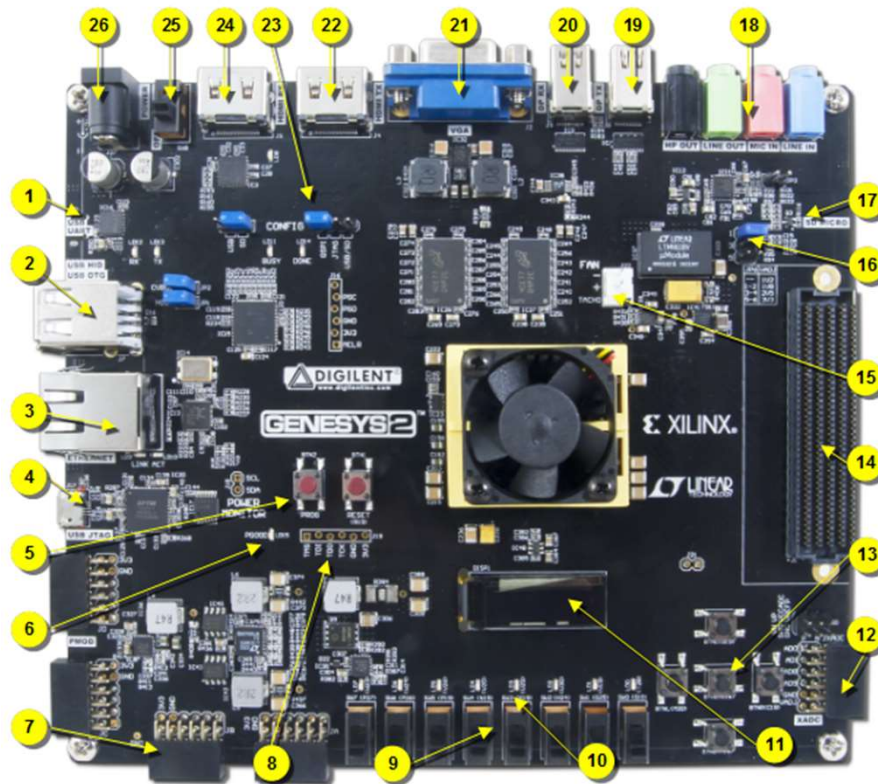


1	Spartan-7 XC7S50-CSGA324 FPGA	10	Power supplies and control
2	1Gbit DDR3 memory (x16)	11	USB host connector and controller
3	Audio output circuit and 1/8" jack	12	Servo motor connectors
4	SD card socket	13	10K Potentiometer (connected to ADC)
5	HDMI source buffer and connector	14	Joystick connector
6	USB power/prog input and FTDI controller	15	GPIO (buttons, switches, LEDs)
7	Power and reset control select	16	Power supply status indicators
8	Optional external power jack (USBC)	17	PDM digital microphone
9	Main power switch	18	FPGA programming status and control

Sistema Embebido

Embebido FPGA

■ Digilent Genesys2 Kintex7 FPGA Board



ID	Component Description
1	USB-UART bridge
2	User-USB (bottom) & USB MSD/HID (top)
3	Ethernet RJ-45 10/100/1000
4	USB-JTAG bridge
5	PROG and user reset buttons
6	Power LED
7	Digital Pmods
8	JTAG header
9	User slide switches
10	User LEDs
11	OLED display
12	Dual analog/digital Pmod
13	User pushbuttons
14	FMC HPC

Sistema Embebido

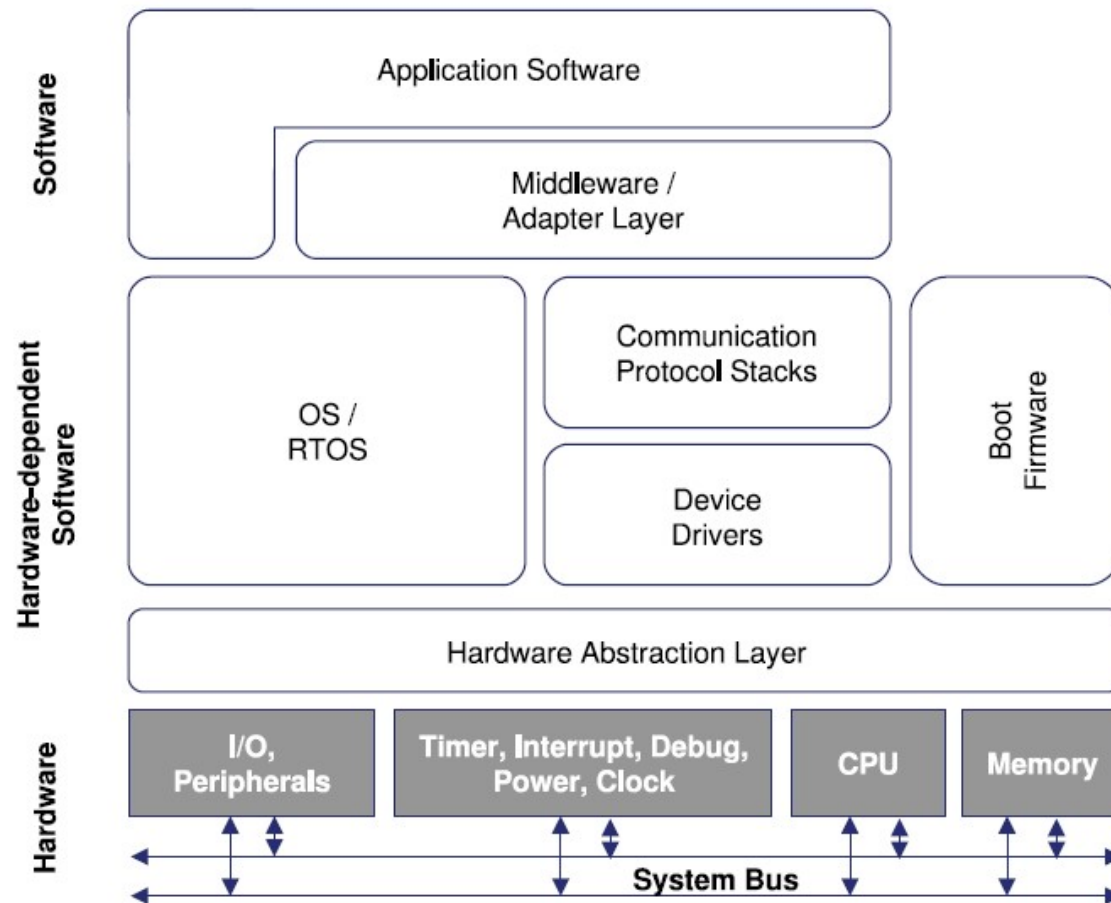
Concepto de Sistema Embebido

- La importancia de la Abstracción
 - Separar conceptualmente los niveles altos de los bajos.
 - Software de bajo nivel que conecta con el hardware o HdS (*Hardware-Dependent Software*)
 - Software de alto nivel que no se preocupa del hardware pero que interconecta con el software de nivel inferior.
- ISA: Instruction Set Architecture.
 - Nivel más bajo de software, comúnmente conocido como ensamblador. Específico de una arquitectura hardware de microcontrolador o microprocesador.
- Sistema Operativo (OS)
 - Encapsula los detalles de acceso a periféricos de I/O, reserva de memoria, etc.
- La combinación de la ISA y del OS crea el concepto de ABI – *Application Binary Interface* -

Layered Software Architecture

Ejemplo Capas

- Ejemplo final de arquitectura con capas software



Layered Software Architecture

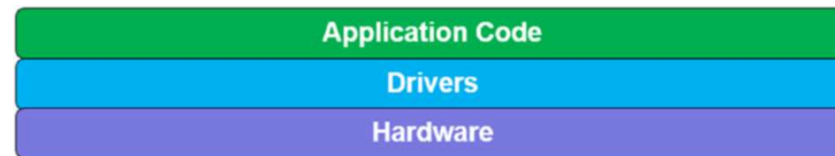
Abstracción SW-HW

- Arquitectura de SW para Embebidos
 - En el principio los recursos limitados de memoria en un microcontrolador hacían necesario crear código no adaptable, sin reuso, pero óptimo para un sistema concreto.
 - Hoy en día los costes decrecientes de memoria y las capacidades de los compiladores permiten la arquitectura SW.
 - Complejidad, Escalabilidad, Portabilidad y Reuso implican el uso de arquitectura software.
 - La capa de Aplicación debe ser independiente del uC (o casi). Esto facilita el reuso.
 - Las capas de software permiten:
 - Limitar las características de cada una y su funcionalidad. Modularizan el desarrollo software.
 - Necesitan crear interfaces entre capas reusables.
 - Ocultan información no necesaria de la capa de aplicación a capas inferiores.

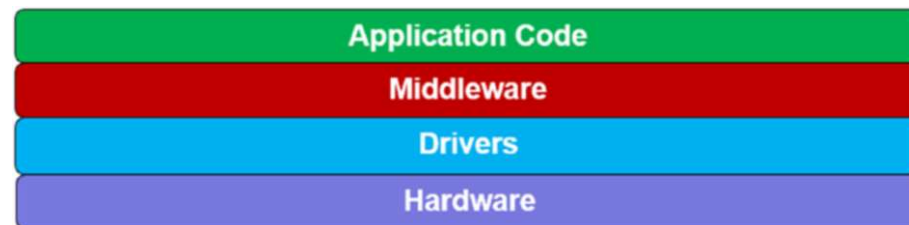
Layered Software Architecture

2 y 3 niveles

- Arquitectura de SW para Embebidos
 - 2 niveles: La más sencilla, drivers conectan con hardware (uC, sensores, I/O, power...), aplicación tiene acceso a LLD (*Low-Level Drivers*) pero los detalles de dicho HW permanecen ocultos.



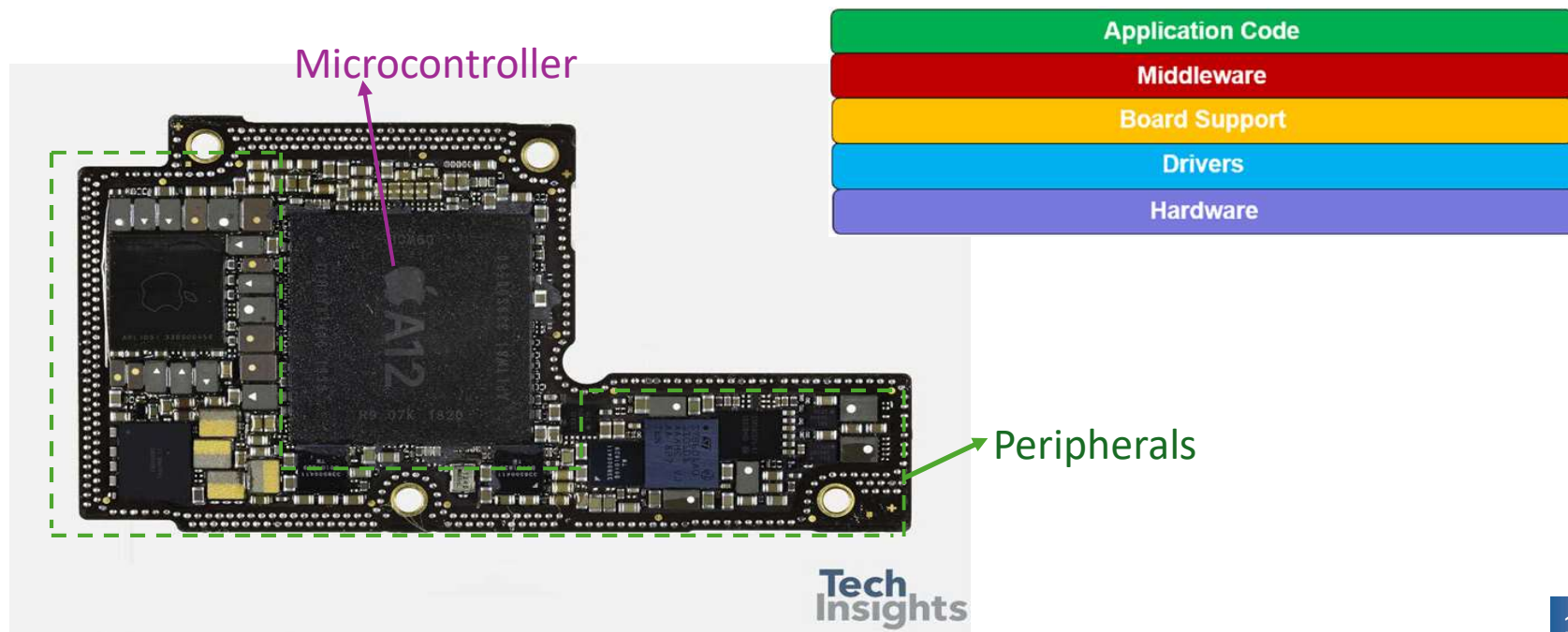
- 3 niveles: Se introduce un Middleware que permite encapsular elementos específicos como RTOS, Stacks USB o Ethernet, Sistema de Ficheros,...



Layered Software Architecture

El BSP

- 4 capas: A menudo el acceso al SW de periféricos se denomina BSP Board Support Package.
- El BSP puede estar separado de los drivers del microcontrolador.



Layered Software Architecture

5 capas - HAL

- Hardware Abstraction Layer (HAL)
- El HAL es un *interface* que proporciona a los programadores de la aplicación un conjunto de funciones de acceso al hardware sin conocer los detalles del mismo.
- Debe ser portable, reusable, abstraer del funcionamiento interno, escalable entre familias del mismo uC.



Layered Software Architecture

6 capas - API

- Application Programming Interfaces (API).
- El API es un conjunto de rutinas, funciones y librerías que aceleran el desarrollo del software de aplicación.
- Las APIs se desarrollan en los niveles más altos del software.
- Hay desarrolladores que denominan al HAL como API ya que permite conectar con el hardware.
- En el middleware elementos como RTOS, TCP/IP Stack pueden tener cada uno su API.



- No es aconsejable crear muchas capas SW.
- Las interfaces entre capas deben estar plenamente definidas.

Layered Software Architecture

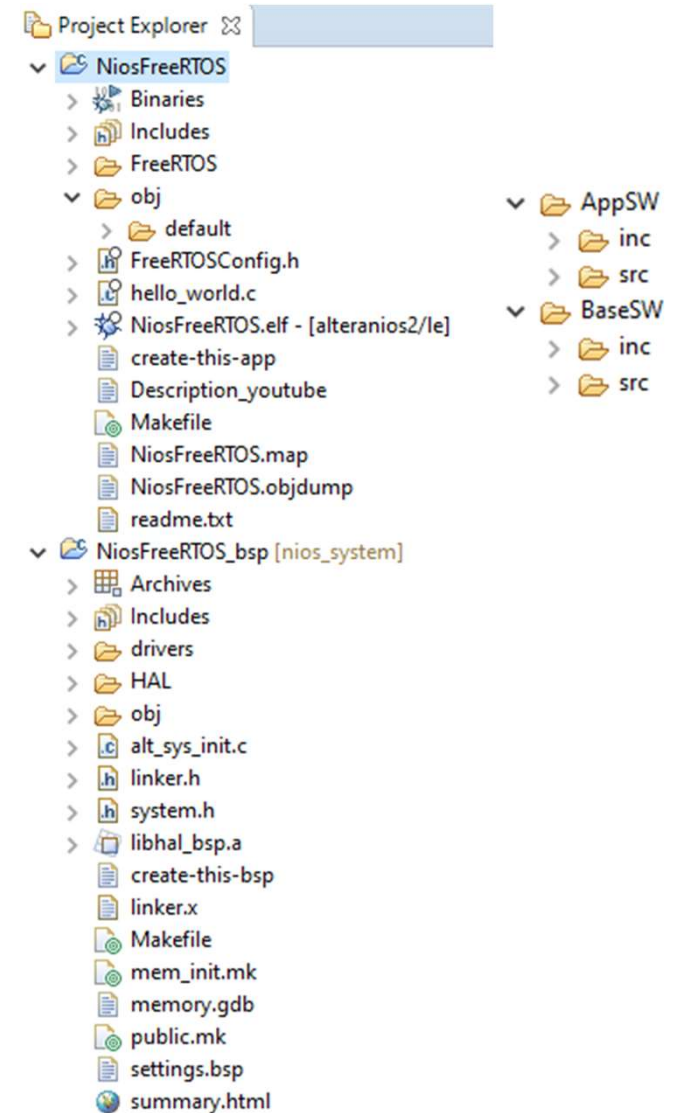
Project organization

- Habitualmente la arquitectura de niveles se distribuye en carpetas.
- Cada carpeta puede representar un nivel.
- En las carpetas se suele distribuir los ficheros de cabecera y los de programa en subcarpetas distintas.
- No hay una norma escrita pero cada organización suele utilizar el mismo sistema de carpetas para distribuir las capas software y hacer el sistema completo portable, reusable y actualizable modularmente.

App



HW



Layered Software Architecture

Project organization

- Habitualmente la arquitectura de niveles se distribuye en carpetas.
- Cada carpeta puede representar un nivel.
- En las carpetas se suele distribuir los ficheros de cabecera y los de programa en subcarpetas distintas.
- No hay una norma escrita pero cada organización suele utilizar el mismo sistema de carpetas para distribuir las capas software y hacer el sistema completo portable, reusable y actualizable modularmente.



App



Middleware

```
▼ NiosFreeRTOS
  > .settings
  ▼ AppSW
    > inc
      task1.h
      task2.h
      task3.h
      task4.h
    > src
      task1.c
      task2.c
      task3.c
      task4.c
  ▼ BaseSW
    > inc
      lcd.h
      led.h
      mtl.h
    > src
      lcd.c
      led.c
      mtl.c
  ▼ FreeRTOS
    > include
    > portable
    croutine.c
    event_groups.c
    list.c
    queue.c
    stream_buffer.c
    tasks.c
    timers.c
    > obj
    .cproject
    .project
    create-this-app
    Description_youtube
    FreeRTOSConfig.h
    hello_world.c
    hello_world.h
    Makefile
    NiosFreeRTOS.map
    readme.txt
```

Drivers



HAL



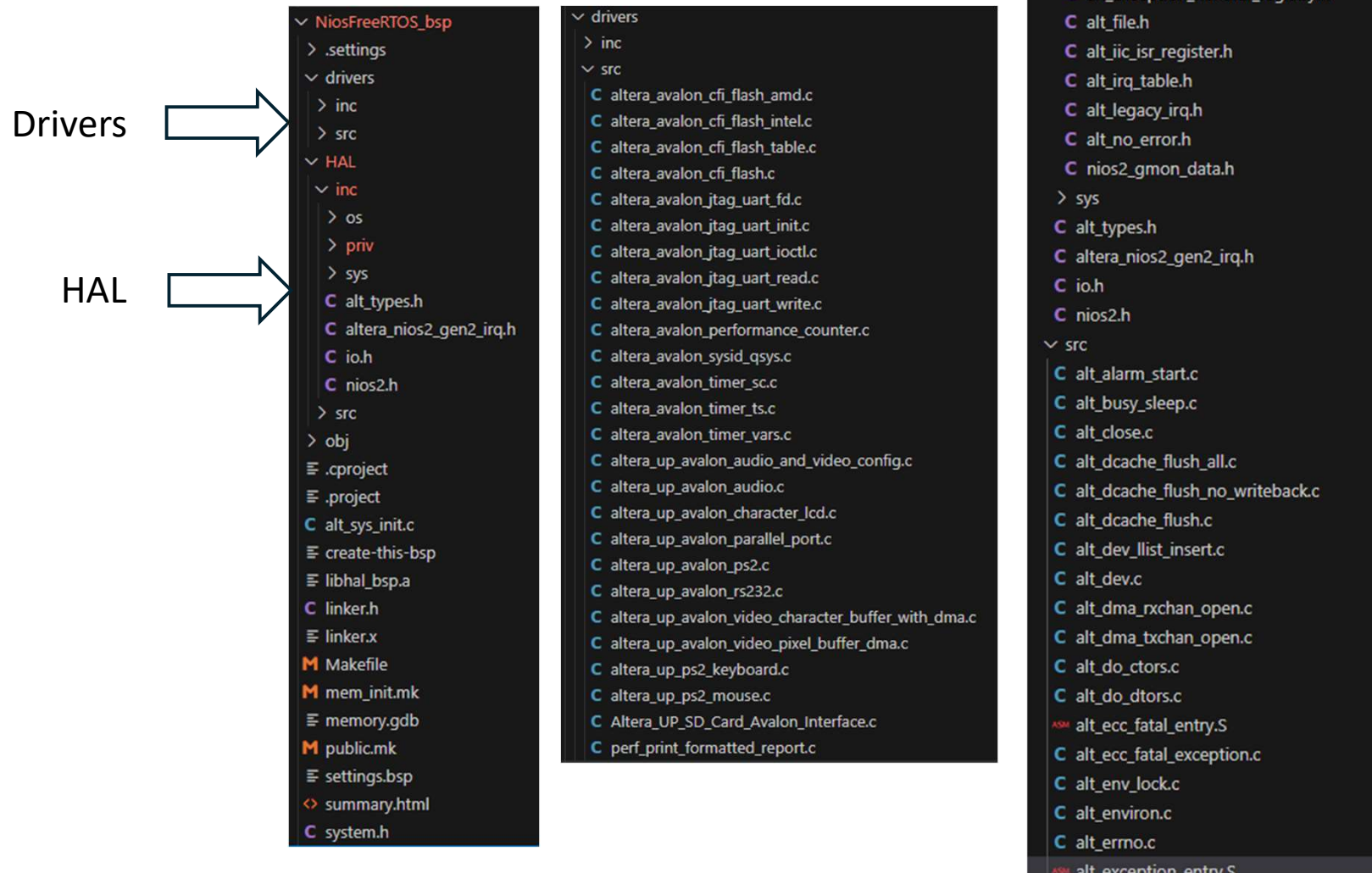
BSP

```
▼ NiosFreeRTOS_bsp
  > .settings
  ▼ drivers
    > inc
    > src
  ▼ HAL
    > inc
      os
      priv
      sys
      alt_types.h
      altera_nios2_gen2_irq.h
      io.h
      nios2.h
    > src
  > obj
  .cproject
  .project
  alt_sys_init.c
  create-this-bsp
  libhal_bsp.a
  linker.h
  linker.x
  Makefile
  mem_init.mk
  memory.gdb
  public.mk
  settings.bsp
  summary.html
  system.h
```

Layered Software Architecture

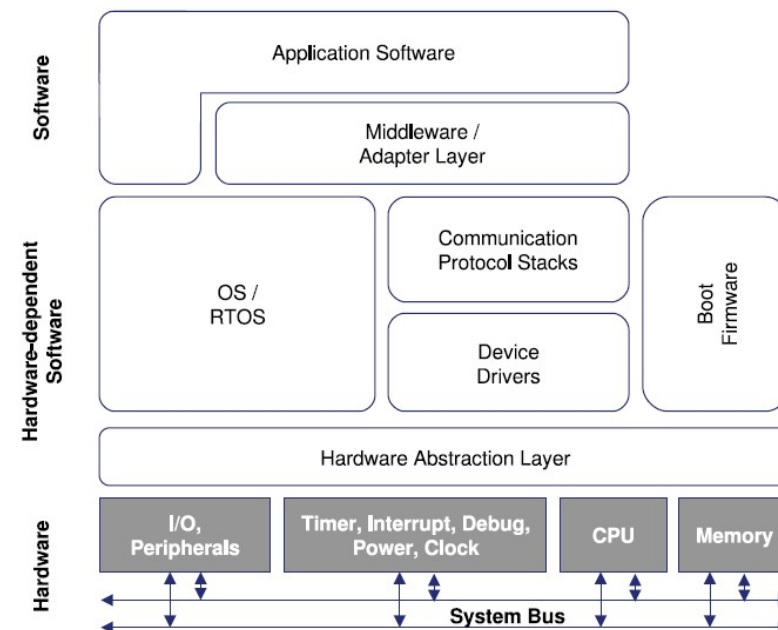
Project organization

■ Ejemplo NIOSII BSP



Complex Layered Software Architecture

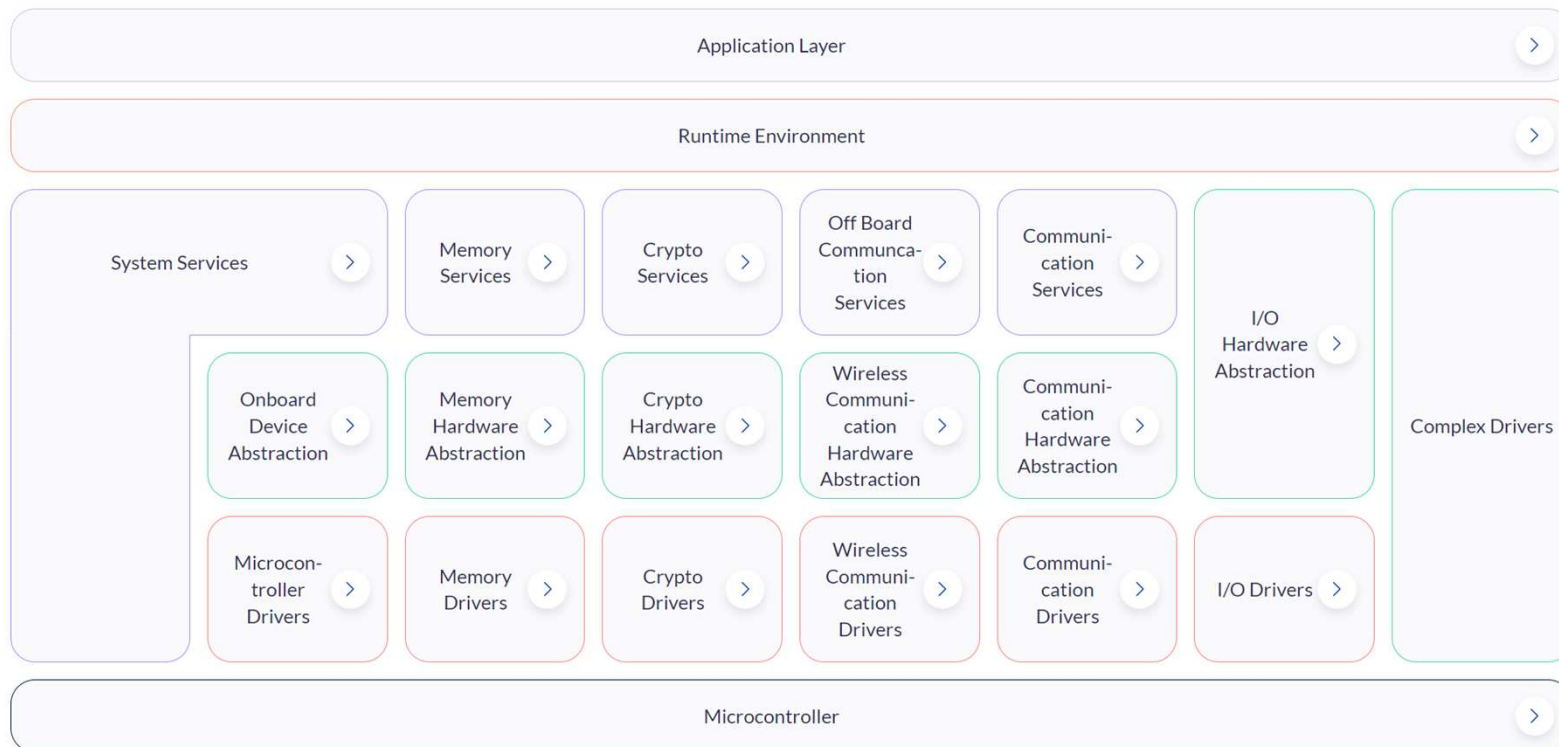
- Las arquitecturas pueden ser muy complejas y las interfaces entre capas no están normalizadas.
- Algunas SI como AUTOSAR. AUTomotive Open System Architecture.
<https://www.autosar.org/>
- AUTOSAR requiere procesadores por encima de 200MHz.



Layered Software Architecture

AUTOSAR

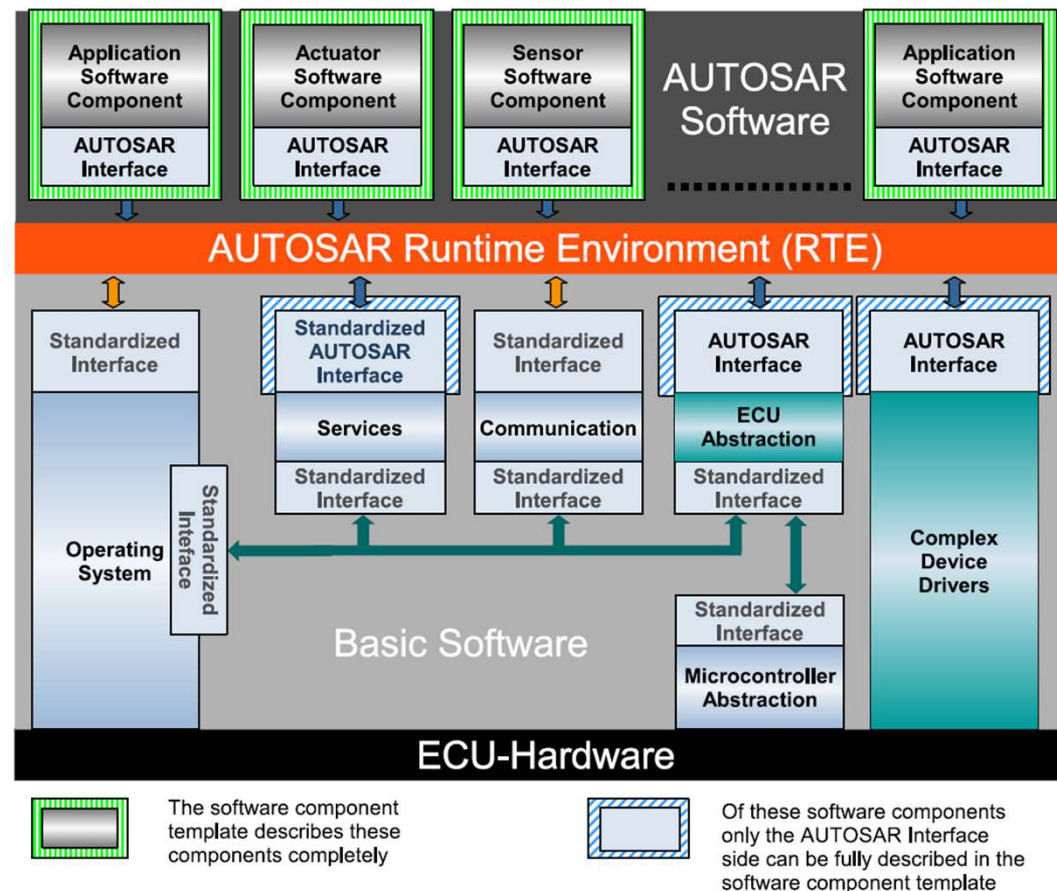
■ AUTOSAR Classic Release



Layered Software Architecture

Autosar Standard

- AUTOSAR ECU Architecture Diagram



Layered Software Architecture

AUTOSAR

- Aquí se puede ampliar con detalles de la presentación de AUTOSAR que nos pasó BMW

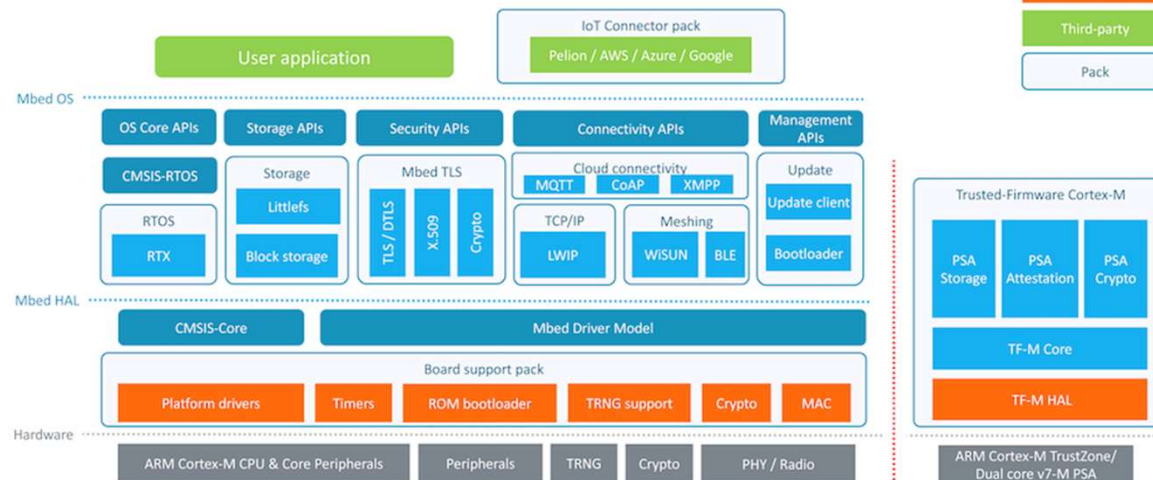
Layered Software Architecture

Estandarización de Arquitecturas o Plataformas

- AUTOSAR es una arquitectura estándar de producción.
- Otros intentos (no de producción) son:
 - Arduino <https://www.arduino.cc/>
 - Mbed-ARM <https://www.mbed.com/en/>

Mbed OS 6 Conceptual Architectural

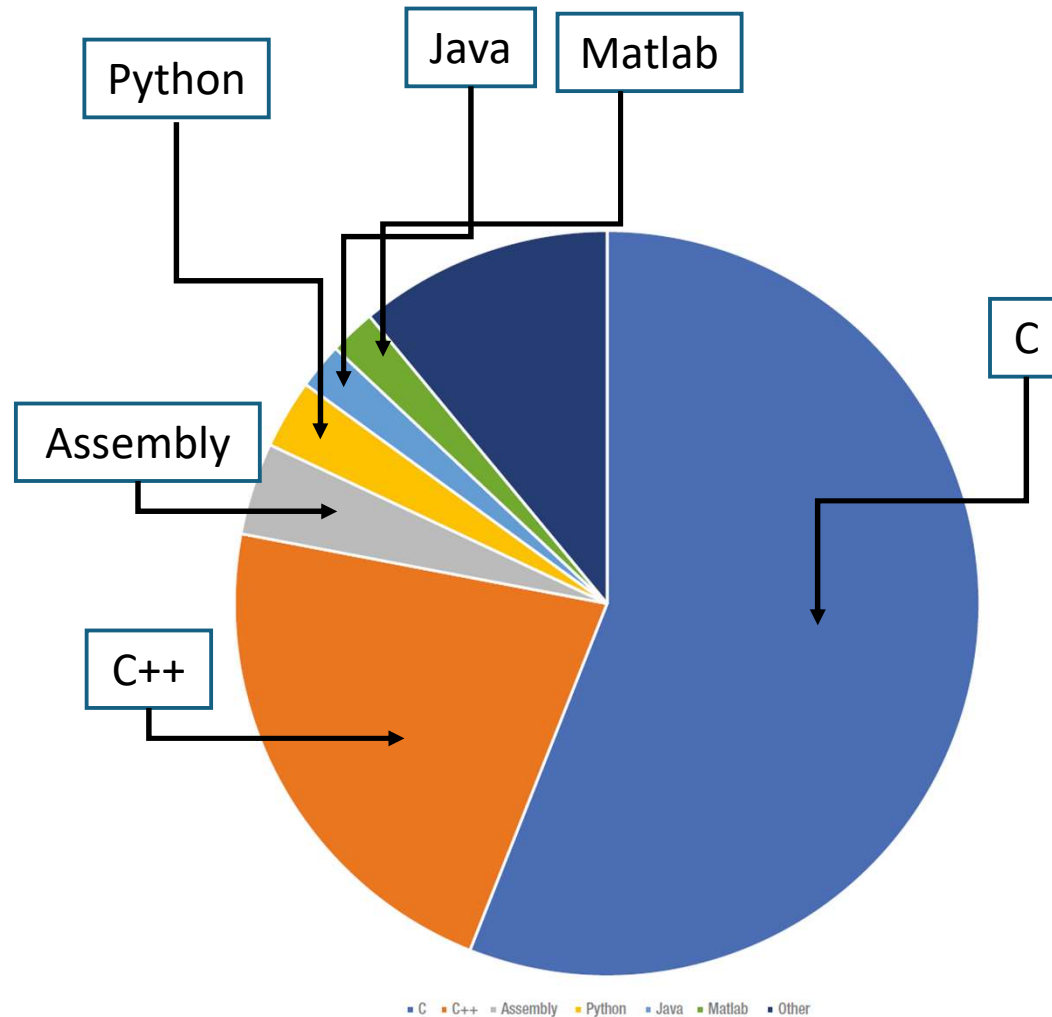
Componentized, Layered Architecture



Lenguajes

C es el lenguaje de sistemas embebidos

- C estándar (ANSI C). Versión C99 la más usada.
- C es cercano al HW (uso de memoria, tamaño de código, manejo de variables).
- Solo ensamblador cuando se requiere alta velocidad y código muy compacto.
- C es un lenguaje muy conocido, permite gestionar recursos de memoria y es fácilmente portable entre diversas plataformas uP de -8, -16, -32 y -64bits.
- MISRA C: estándar de automoción muy usado de forma general-> recomendaciones sobre un subset de C.



<https://www.techonline.com/tech-papers/a-simple-style-for-c-programming/>

Lenguajes

Herramientas y Normas

- MISRA-C (<https://misra.org.uk/>)
- Linter para MISRA-C (<https://pclintplus.com/>)
- Documentación de código
(<https://www.doxygen.nl/>)
- Figura ejemplo de cabecera Doxygen

Portabilidad

- El reuso de SW cuando se cambia de hardware es fundamental.
- La portabilidad implica utilizar el mismo software de aplicación sobre diversas plataformas hardware.
- Reduce los esfuerzos de diseño SW, pero se debe asegurar la migración o adaptación del software sobre la nueva plataforma.
- Es habitual en SoC design, tanto en MPSoC (MultiProcessor SoC), donde conviven:
 - DSP para procesamiento de señal específico.
 - uC para aplicaciones intensivas de control.
 - ASIP (*Application Specific Instruction Set Processors*), CPU específica para una aplicación concreta.
- Hay que centrarse en dos grandes bloques de diseño SW:
 - HW Independent: abstracción y reuso.
 - Application SW
 - OS. *Operating System*
 - HW Dependent: portabilidad
 - HAL. *Hardware Abstraction Layer*. Define la portabilidad.
 - Drivers. No portables.

Portabilidad

■ Portabilidad en C

- El estándar no define tipos de datos integer, dependen de la arquitectura HW.

```
static int i = 0;
static uint32_t LoopCount = 0;

for (i = 1; i != 0; i++)
{
    LoopCount++;
}
```

¿Cuál será el valor de LoopCount cuando i alcance el valor 0?

- a) 65.535
- b) 4.294.967.295

- Depende del compilador
- Solución: uso de stdint.h

Data Type	Minimum Value	Maximum Value
int8_t	-128	127
uint8_t	0	255
int16_t	-32,768	32,767
uint16_t	0	65535
int32_t	-2,147,483,648	2,147,483,647
uint32_t	0	4,294,967,295

Hardware Abstraction Layer

HAL

- Puede incluir o no los drivers
- Es la capa de adaptación al HW
- Depende del fabricante de HW
- Ejemplo muy sencillo: RX-TX serie

TX_RX HW

- Registros físicos de la IP (diseño RTL)

Register	AccessExt	Offset	Width	AddrUnit
SIF_TXD_REG	R	0	32	32
SIF_RXD_REG	W	0	32	32
SIF_FLAG_REG	R	1	32	32
SIF_CFG_REG	W	1	32	32

- Fichero io_ports_reg.h

```
/**
 * @file
 * @brief Macros for memory addresses.
 *
 * This header file contains macros that initialize references to readable/writable registers in Transmitter Receiver.
 * The macros are used to access the registers in the SIF module.
 * SIF_TXD_REG and SIF_RXD_REG are used to access the TXD and RXD registers, respectively. But they share the same memory address.
 * SIF_TXD_STATUS_REG and SIF_RXD_STATUS_REG are used to access the TXD_STATUS FLAG and RXD_STATUS FLAG, respectively.
 * The status flags indicate if the TXD and RXD registers are ready to be written to or read from.
 */

#define SIF_TXD_REG 0xFF000000
#define SIF_RXD_REG 0xFF000000
#define SIF_TXD_STATUS_REG 0xFF000004
#define SIF_RXD_STATUS_REG 0xFF000008
```

HAL

TX_RX Simple

- Funciones HAL
- Uso de Funciones en App

```
// reading a value from serial stream
alt_u32 rx = read_from_serial_stream();

//writing a value to serial
streamwrite_to_serial_stream(0x00000001);
```

```
#include "io_port_regs.h"
// alt_u32 equivalent to uint32_t
volatile alt_u32 *rx_reg_ptr,
                 *tx_reg_ptr,
                 *data_transmitted_ptr,
                 *data_received_ptr;

struct flag_register_t {
    volatile alt_u32 data_transmitted : 1;
    volatile alt_u32 data_received : 1;
    const alt_u32 unused : 30;
};

void initialize_registers() {
    /* Initialize references to readable registers */
    rx_reg_ptr = (alt_u32 *) SIF_TXD_REG;
    data_transmitted_ptr = (alt_u32 *) SIF_TXD_STATUS_REG;
    data_received_ptr = (alt_u32 *) SIF_RXD_STATUS_REG;
}

/* ready flags*/
alt_u32 is_ready_for_transmission() {
    return (alt_u32)((volatile alt_u32 *) (SIF_TXD_REG) & 0x00000001);
}
alt_u32 is_data_ready() {
    return (alt_u32)((volatile alt_u32 *) (SIF_TXD_REG) >> 1) & 0x00000001);
}
/* writing a value to serial stream checking if flag is set*/
void write_to_serial_stream(alt_u32 data) {
    if (is_ready_for_transmission()) {
        tx_reg_ptr = (alt_u32 *) SIF_TXD_REG;
        *tx_reg_ptr = data;
    } else {
        // handle error
    }
}

/* reading a value from serial stream*/
alt_u32 read_from_serial_stream() {
    if (is_data_ready()) {
        rx_reg_ptr = (alt_u32 *) SIF_RXD_REG;
        return *rx_reg_ptr;
    } else {
        // handle error
    }
}
```

Real Time Operating Systems

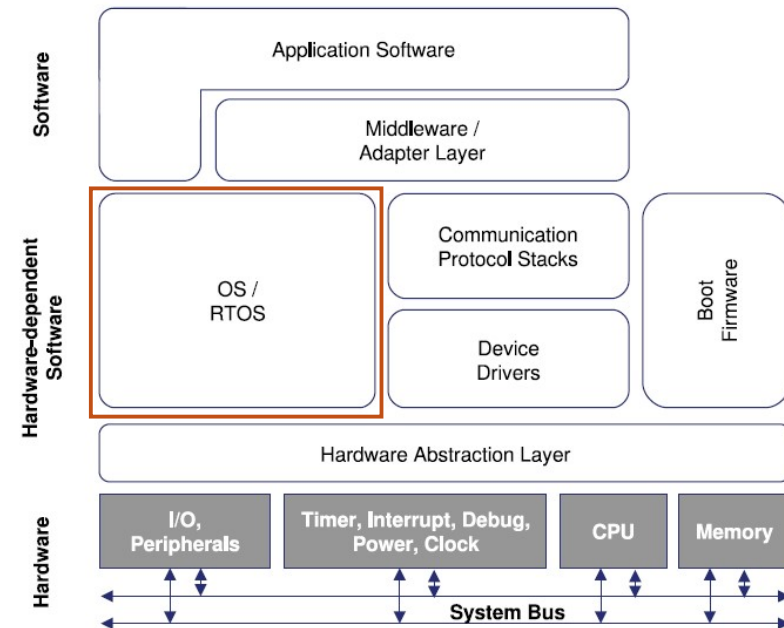
RTOS

- Introducción
- Ejemplos
- Conceptos y Definiciones
 - Tareas
 - Scheduler
 - Comunicación entre Tareas
 - Expulsivos
 - Deadlock
 - Cambio de Contexto
 - Función Reentrante

Introducción RTOS

Definición

- Un **RTOS** es un OS específico **para aplicaciones embebidas**, es decir para hardware con recursos limitados.
- Un OS general está diseñado para HW sin limitaciones o restricciones.
- Un OS es un *component software* responsable de coordinar la ejecución concurrente de diversas tareas mediante:
 - Gestión de los Recursos HW (processor, memory, access to peripherals,)
 - Garantizar un conjunto de Servicios (communication, synchronization, .)
- Un OS se implementa como un kernel (un programa autónomo), junto con una librería de funciones (API del OS) para acceder a sus servicios de forma controlada.



Introducción RTOS

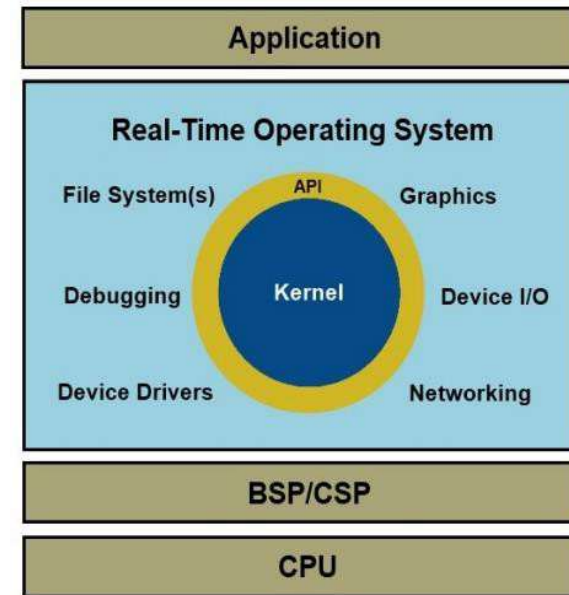
Definición

- Tiempo Real (RT)
 - Realizar cálculos o resolver problemas con unas determinadas restricciones temporales (*deadline*)
 - Un sistema no tiene que procesar todos los datos instantáneamente o muy rápido para ser considerado de tiempo real
- Clasificación Sistemas en Tiempo Real (RTS)
 - **Soft** Real-Time System (RTS no-críticos con plazo flexible)
 - Un fallo en el cumplimiento de los requerimientos de tiempo real degrada prestaciones pero no es fatal. [navegador web, decodificador MPEG...]
 - **Firm** Real-Time System (RTS no-críticos con plazo firme)
 - Los incumplimientos del deadline hacen que el resultado como producto sea inservible pero no daña o destruye nada. [sistema videoconferencia, control de navegación de una cosechadora autónoma, predictor meteorológico, ...]
 - **Hard** Real-Time System (RTS Críticos)
 - Un fallo de incluso uno de los requerimientos puede provocar un fallo catastrófico del sistema. [airbag, control de un *Traction Inverter*, *Power-Train Control*, *Control de Frenos*...]

Introducción RTOS

Componentes RTOS

- Administrador de tareas (*task management*)
 - Asignación de recursos de procesamiento
 - Sincronización
- Manejo de memoria
 - Asignación
 - Liberación y consolidación (*garbage collection*).
- Manejo de interrupciones
- Manejo de entrada/salida: conexión HAL – Drivers
- Recursos de comunicaciones: Ethernet, USB, ZigBee, ..(deseable)
- Otros:
 - Entorno de desarrollo
 - Compatibilidad con múltiples BSP



Introducción RTOS

Ejemplos RTOS

■ Hay multitud de RTOS

- VxWorks/eCos / Nucleus / QNX / Windows Phone / iOS / Android / LynxOS / RTLinux / QNX /
- **FreeRTOS**
- **MicroC - OS II / MicroC - OS III/**
- **Zephyr**
- **Autosar/OSEK**
- SafeRTOS
- Integrity
- embOS
- TI-RTOS (Texas)
- CMSIS-RTOS (microchip Keil)

AUTOSAR/OSEK

- Especifico de Automoción.
- Standard multiplataforma.
- Estándar en seguridad.
- Muy Caro.

UCOS

- Funciones Claras y bien documentado.
- Ejemplo docente.
- Tendencia a la baja.
- Gratuito.

FreeRTOS

- Muy extendido
- Gratuito
- Portabilidad asegurada

Zephyr

- Estilo devicetree Linux para crear la configuración del sistema
- Actual y tendencia

CMSIS

- Wrapper de FreeRTOS
- Cortex M

Introducción RTOS

Ejemplos de RTOS

- FreeRTOS
 - <https://www.freertos.org/>
- UCOS
 - <https://www.silabs.com/developers/micrium-os>
- Zephyr
 - <https://zephyrproject.org/>

Introducción RTOS

Características RTOS

- Pequeño tamaño (*ROMable*)
- Tolerante a fallos: debe ejecutarse mucho tiempo sin intervención humana
- Bajo consumo de potencia (*SleepMode*)
- Capaces de rearrancar autónomamente
- Soporte a configuración remota (*Boot - FOTA*)

Introducción RTOS

Ejemplo RTOS básico

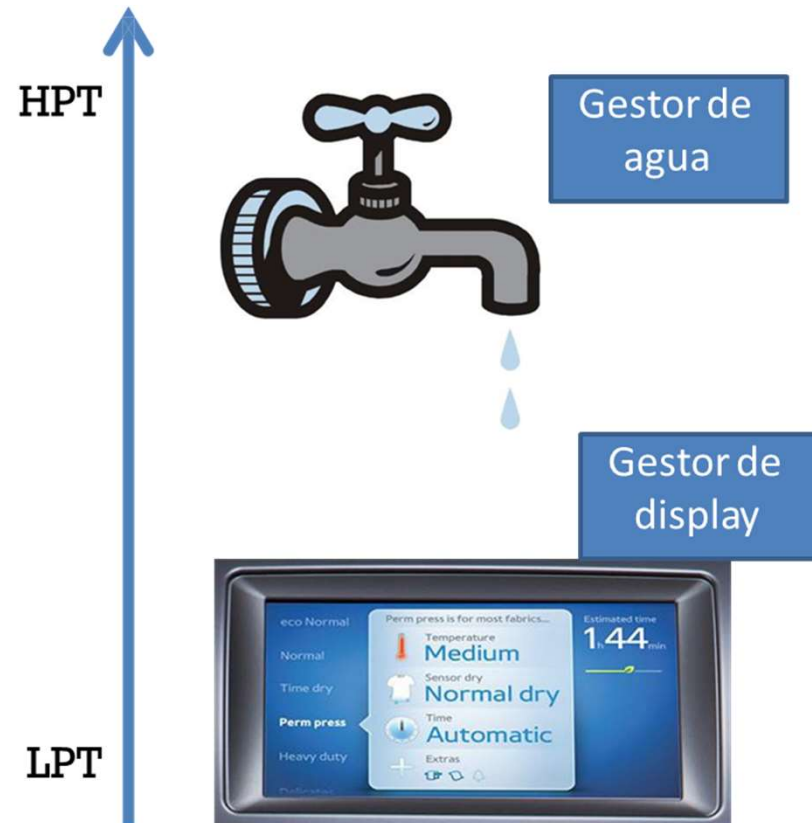
- Controlador de Lavadora, tareas a realizar:
 - Toma y evacua agua
 - Control de motor
 - Apertura y cierre de puerta
 - Lectura de mandos de usuario
 - Visualización en *display*
- Existe un único procesador que debe ejecutar las tareas en "paralelo" . Debe dar esa apariencia.
- Los tiempos de respuesta de las tareas deben permitir cumplir con los *deadlines* establecidos.
- No hacerlo puede resultar catastrófico.



Introducción RTOS

Ejemplo RTOS básico

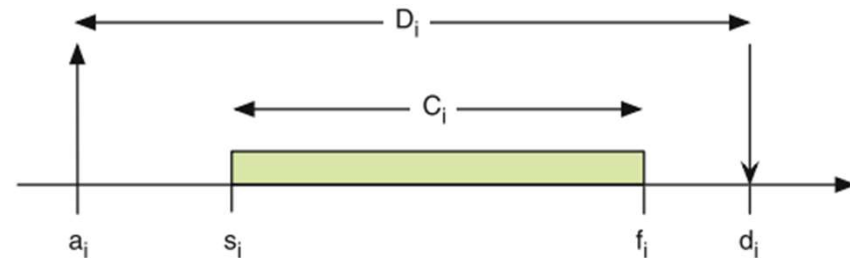
- La lavadora y las prioridades
- Cada tarea tiene una prioridad
- Determina el orden relativo de importancia de las tareas.
 - HPT: High Priority Task
 - LPT: Low Priority Task
- En algunos RTOS las task pueden compartir prioridad pero siempre una se ejecutará antes que otra.



Introducción RTOS

Tarea

- Cada Tarea (J_i) estará definida por ciertos parámetros temporales.
- a_i : arrival time. Tiempo en el que la tarea está *ready* o preparada para ser ejecutada por la CPU.
- C_i : computation time
- D_i : deadline relativo
- d_i : deadline absoluto
- f_i : finishing time
- s_i : start time. Tiempo en el que el RTOS decide poner en marcha la ejecución de la tarea.



Introducción RTOS

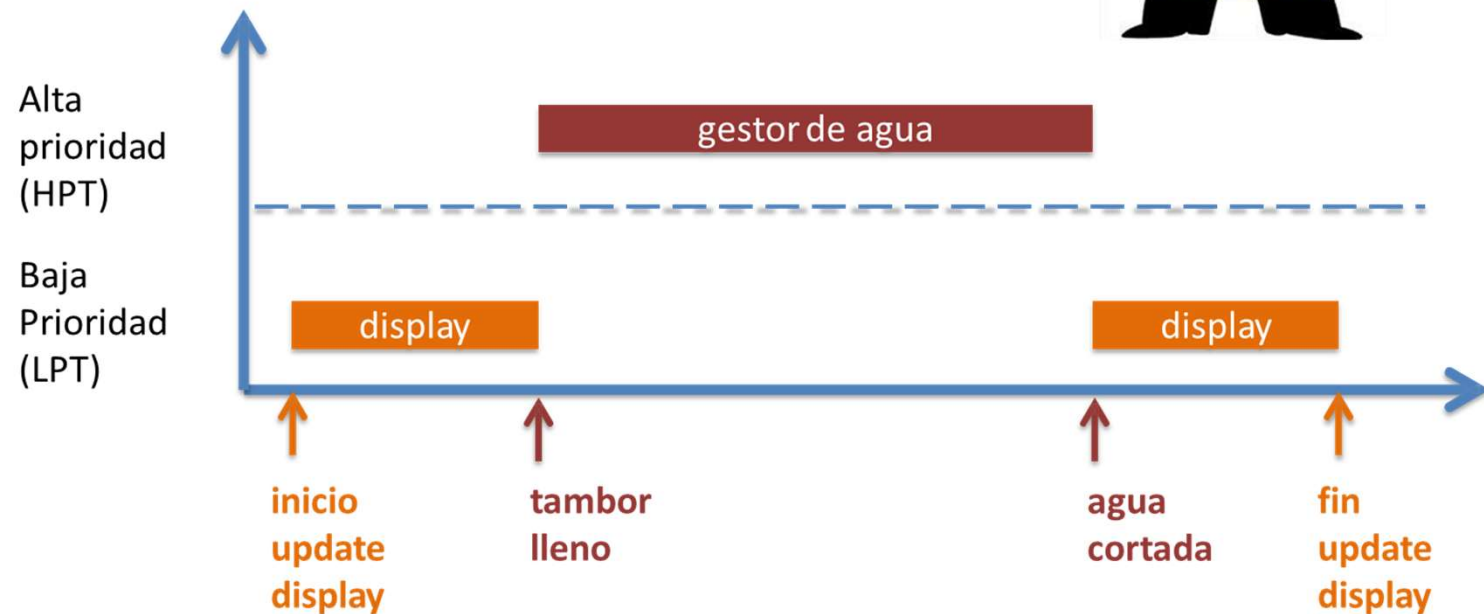
Tarea

- El proceso o tarea
 - Abstracción de un programa en ejecución
 - Unidad lógica de trabajo planificable por el S.O.
 - Puede ser pequeña o grande (Aplicación completa o Component Software en Autosar).
- Representado por estructura de datos privados, el contexto:
 - Identificador: Nivel de prioridad
 - Estado de ejecución (En ejecución, listo, suspendido...)
 - Recursos asociados (registros, pila...)
- Multitarea (**multitasking**): crea la ilusión de que múltiples procesos se ejecutan simultáneamente

Introducción RTOS

Scheduler

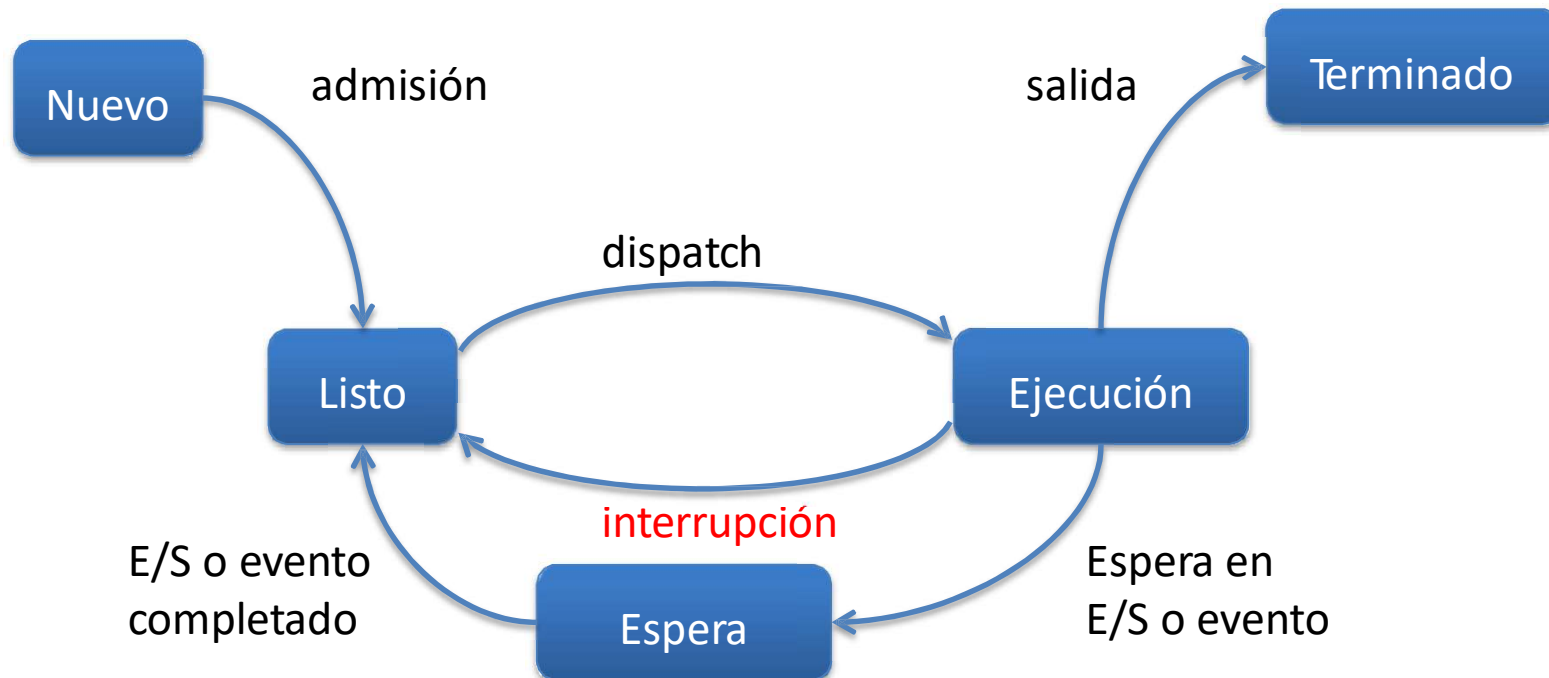
- El planificador (scheduler) decide cuando una tarea se ejecuta en la CPU.
- Los schedulers de un RTOS pueden ser expulsivos (*preemptive*)



Introducción RTOS

Scheduler

- Estados de una tarea, gestionados por el *scheduler*.



Introducción RTOS

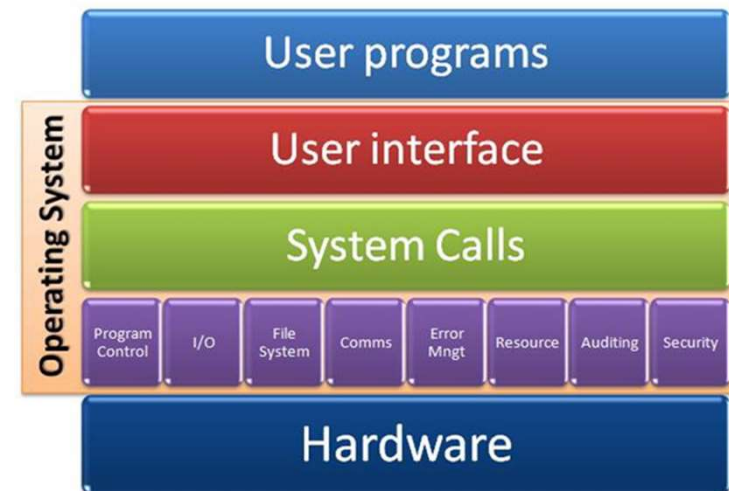
Scheduler

- Existe la posibilidad de RTOS con *scheduler expulsivo* Round-Robin.
 - dentro de un OS se asigna a cada proceso una porción de tiempo equitativa y ordenada, tratando a todos los procesos con la misma prioridad)
 - Se expulsa a la tarea por tiempo (no por prioridad asignada)
- Las prioridades pueden ser estáticas o dinámicas
 - Por ejemplo en el uso de MUTEX
- Pueden sufrir problemas de inanición (*starvation*): hay tareas consumidoras de recursos y otras que no pueden consumir.
- Hay muchas estrategias de scheduling.

Introducción RTOS

Scheduler

- Funciones esenciales para procesos en el *kernel* del **RTOS**
 - Planificador (scheduler): selecciona un proceso para ejecución.
 - Dispatcher: asigna CPU a un proceso ejecutando el cambio de contexto.
 - Comunicación entre procesos y sincronización: permite que tareas paralelas colaboren, gestiona la compartición de recursos. (evitar *starvation*)



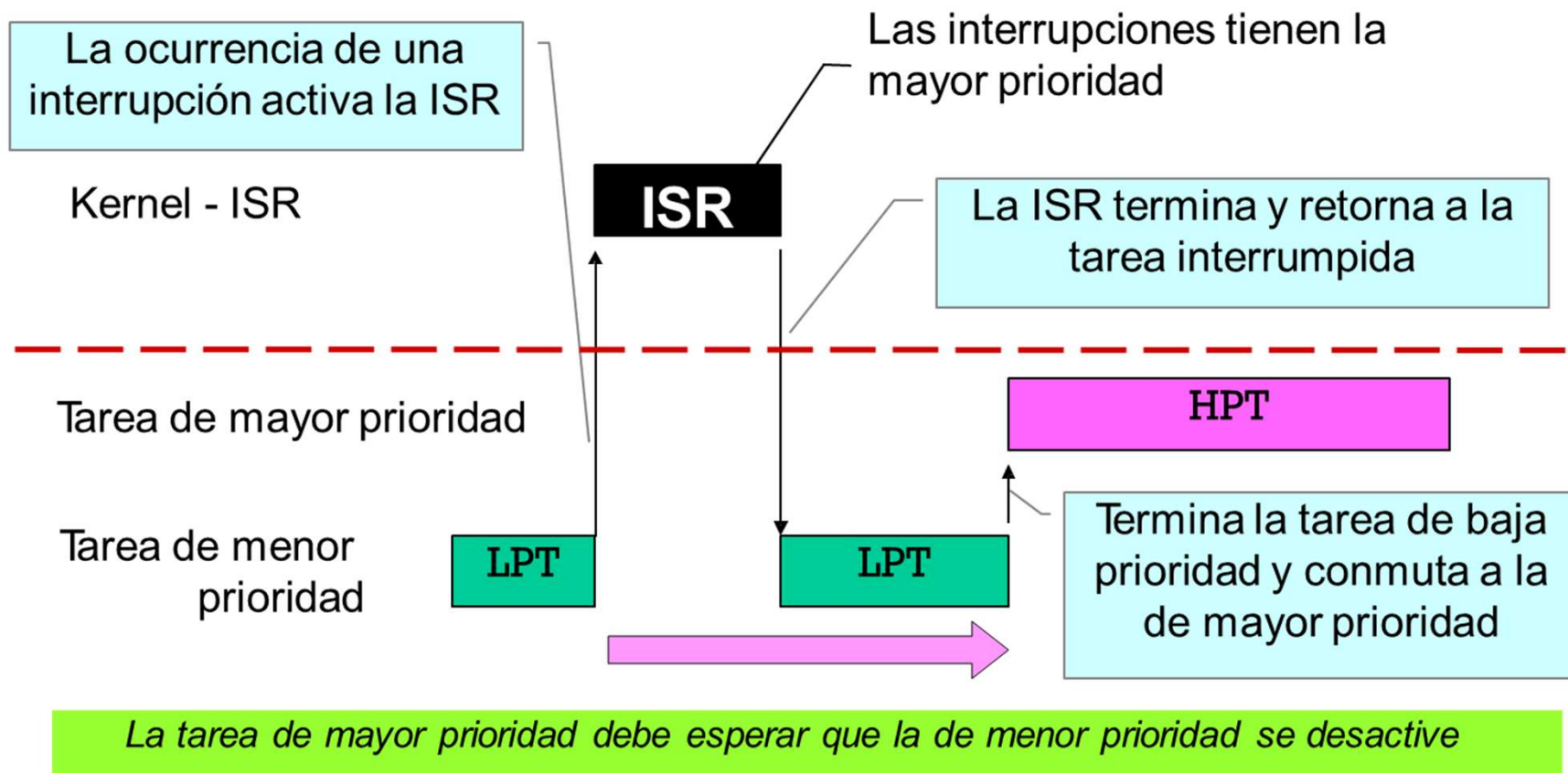
Introducción RTOS

Scheduler

- Estrategias de scheduling.
 - Un ejemplo son los “rate-monotonic systems” (RMS) donde las prioridades más altas se asignan a los eventos más frecuentes. Estos conceptos no son objeto del curso.
- Política definida por:
 - Quién: Selección del proceso que entrará a ejecución
 - Prioridades
 - FCFS (First Come, First Served)
 - SJF (Shortest Job First)
 - Cuándo: momento en que se lleva a cabo la planificación
 - Políticas no expulsivas (non preemptive)
 - Se espera a que el proceso en ejecución se bloquee liberando la CPU
 - Políticas expulsivas (preemptive):
 - Por eventos (sistemas de tiempo real)
 - Por tiempo (sistemas de tiempo compartido). Ej. round-robin

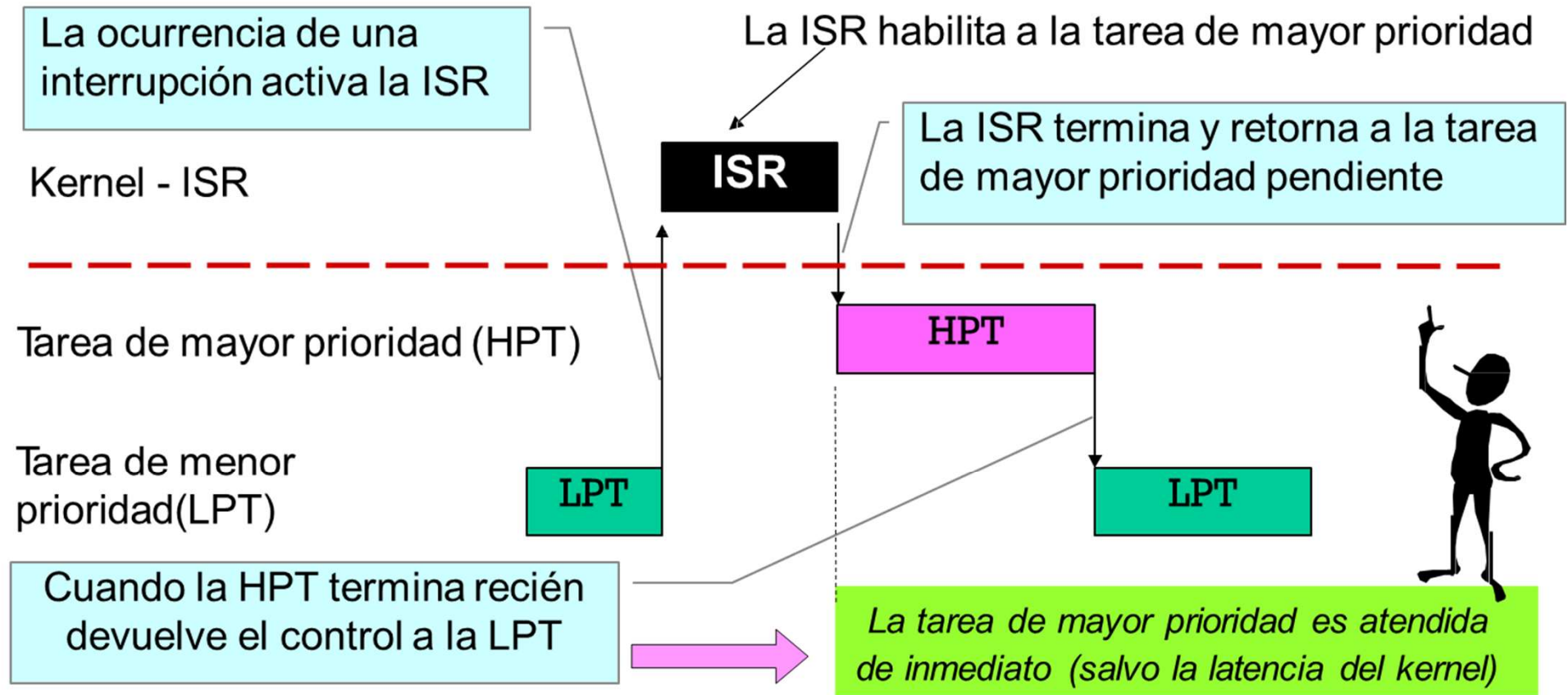
Introducción RTOS

Scheduler no expulsivo (non-preemptive)



Introducción RTOS

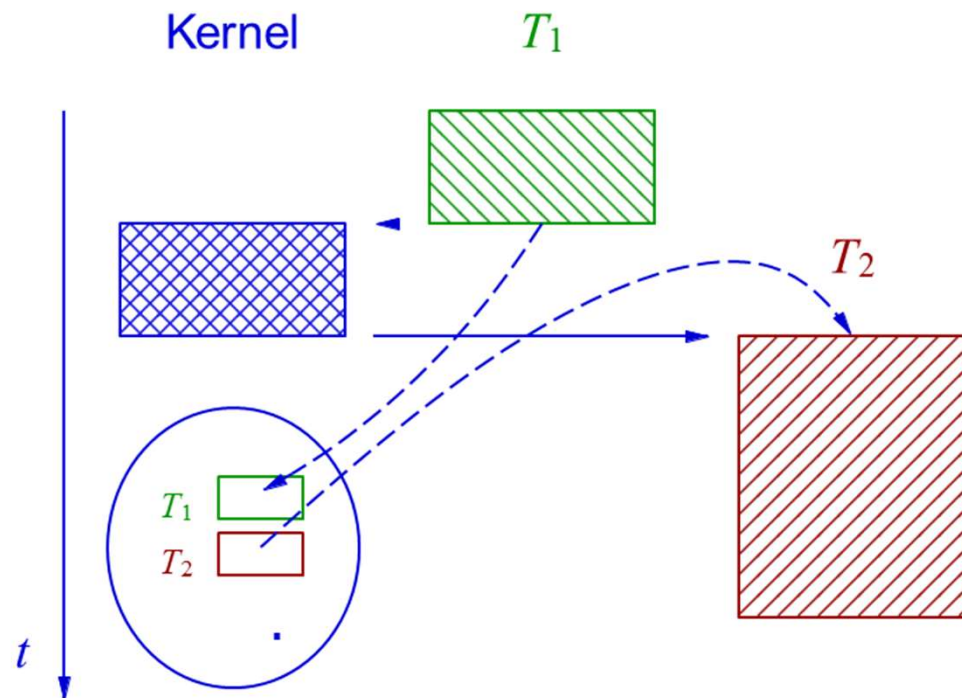
Scheduler expulsivo



Introducción RTOS

Cambio de Contexto

- El *scheduler* transfiere el procesador de un proceso a otro.
- La tarea (*task*) suspendida debe ser capaz de recuperar su estado de ejecución mas tarde. Por ello, el estado del procesador debe salvarse cuando la tarea se suspende.
- La memoria del *kernel* mantiene para cada proceso que no esté en estado durmiente (activo, en ejecución, interrumpido) un área de almacenamiento del contexto.



Introducción RTOS

Cambio de Contexto

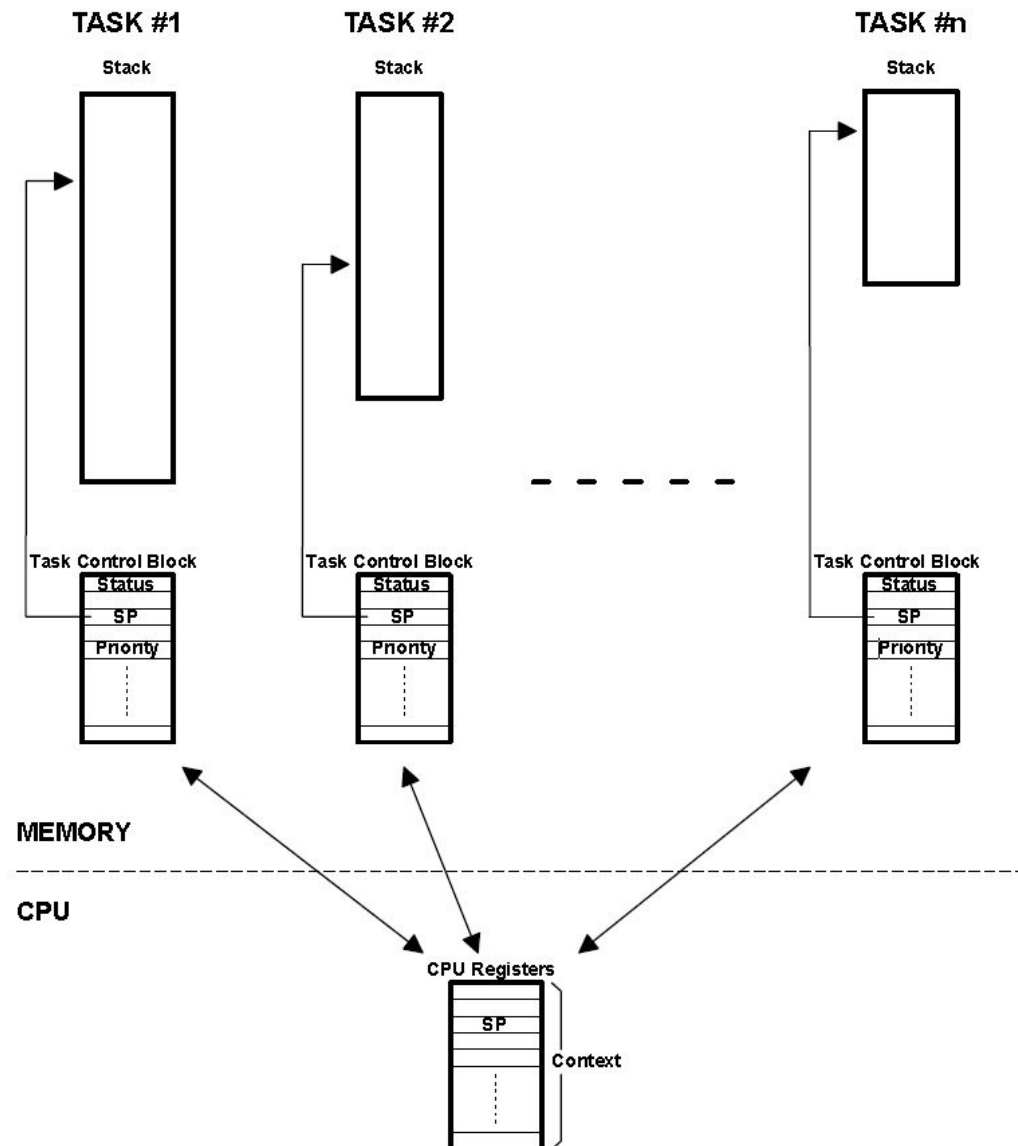
- Los datos de la tarea en ejecución y posteriormente suspendida deben guardarse en la pila de la tarea que contiene:
 - El contexto (dirección de retorno, valores de registros de la cpu) de las funciones en ejecución y
 - Argumentos y variables locales de dichas funciones en ejecución
- La solución a esta forma continua de salvar datos en pila es que cada tarea maneje su propia pila (y por tanto tenga reservado un espacio de memoria).
- En la pila de cada tarea se almacenan los punteros de pila y los registros de cpu necesarios en cada momento, así como las variables y otros datos necesarios.
- Incluso el kernel (scheduler, dispatcher...) maneja su propia pila.

Introducción RTOS

Cambio de Contexto

■ Ejemplo

- Uso de stack por tarea
- TCB
- Gestión por el kernel del RTOS
- Limitar número de tareas



Introducción RTOS

Planificador de Tareas

- Dispatcher: guarda el contexto de la tarea actual y lo sustituye por la tarea entrante.
- Cada tarea requiere un bloque de memoria. Esto limita la cantidad de tareas simultaneas en el sistema.
- Los cambios de contexto son transparentes a la tarea y no agregan trabajo al programador.
- ¡Cuidado con el número de tareas y las comunicaciones entre ellas que generan cambios de contexto!

Introducción RTOS

Función Reentrante

- En un kernel expulsivo –preemptive- la llamada a la misma función desde diversas tareas puede ser problemático.

Variable global (visible en este fichero, donde está definida la función swap)

`static int aux;`

`void swap(int *p1, int *p2)`

```
{  
  aux = *p1;  
  *p1 = *p2;  
  *p2 = aux;  
}
```

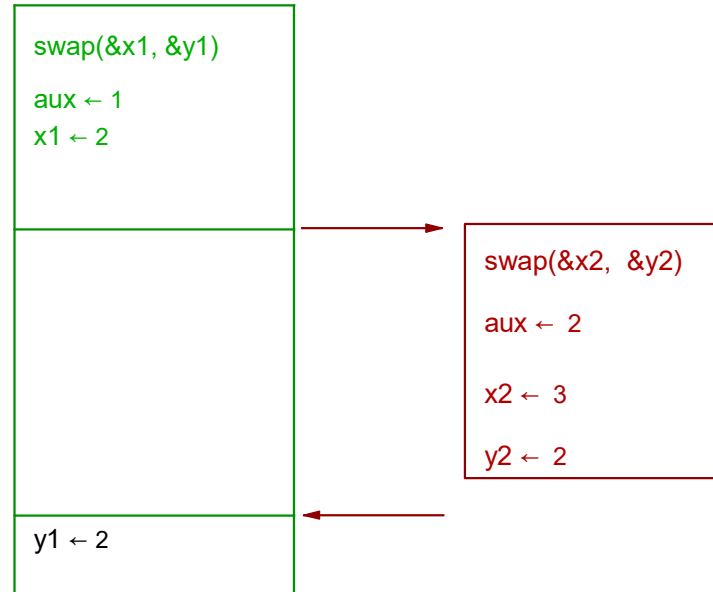
aux

x1

y1

x2

y2



Introducción RTOS

Función Reentrante

- Una función es reentrante cuando puede ser llamada por varias tareas sin posibilidad de conflicto.

```
void swap(int *p1, int *p2)
{
    int aux;

    aux = *p1;
    *p1 = *p2;
    *p2 = aux;
}
```

- Función no-reentrante
 - is_new, test y asignación en instrucciones distintas.
 - printf, puede no ser reentrante (suele serlo y hay que proteger su acceso)

```
volatile int is_new; /* modified by another task */
void display(int v)
{
    if (is_new)
    {
        printf(" %d", v); is_new = 0;
    }
    else
        printf(" ---");
}
```

Introducción RTOS

Características de los RTOS (II)

- Un RTOS debe tener muy documentadas:
 - La estrategia de scheduling,
 - El máximo tiempo de ejecución de cada servicio (DETERMINISMO)
 - Cada mecanismo interno que pueda afectar a la latencia (intervalo máximo de tiempo que las interrupciones están deshabilitadas por el kernel)
- El usuario puede implementar operaciones urgentes como ISR (*Interrupt Service Routine*)
- El RTOS proporciona servicios con gestión de tiempo: *timers* periódicos, tareas de ejecución periódicas...
- Sin embargo, pueden no existir mecanismos de protección contra código de usuario inválido
- La configuración del *kernel* del RTOS puede ser parametrizada con cierto detalle por el usuario

Introducción RTOS

Sistemas Embebidos sin RTOS

- En muchas ocasiones pueden implementarse sin sistema operativo
 - Bucle infinito de tareas
 - Basados en interrupciones (*interrupt-only systems*)
 - Mezcla de los previos: *Foreground/background Systems*
- Es la forma de programar un embebido sin RTOS (normalmente se conoce como ***baremetal*** o *baremetal C* en caso de explicitar el lenguaje de programación)
- Son sistemas sin administrador de tareas

Introducción RTOS

Bucle infinito de tareas

- Ilusión de simultaneidad con tareas cortas en un procesador rápido
- Adecuado solo para sistemas muy sencillos por la dificultad de dividir uniformemente las tareas

```
for(;;) {                                /* bucle infinito A */
    task_1();
    task_2();
    ...
    task_n();
}

for(;;) {                                /* bucle infinito B */
    task_1();
    task_2();
    ...
    task_n();
    task_2();                             /* diferentes ritmos */
}
```

Introducción RTOS

Interrupt-Only Systems

- El programa principal no hace nada
- Todas las tareas son planificadas vía interrupciones y despachadas vía subrutinas de tratamiento de interrupciones
- A menudo se denominan **Runnables** a cada ISR (R1=20ms, R2=10ms, R3=1ms, R4=0,5ms)
- En cada *runnable* se llaman a las funciones que se desean ejecutar en ese instante

```
void main(void)
{
    init();          /* ini. sist. */
    while(TRUE);
}
void int_1(void) /* int hand 1 */
{
    save(context); /* to stack */
    task_1();      /* exec. task */
    restore(context);
}
void int_2(void) /* int hand 2 */
{
    save(context);
    task_2();
    restore(context);
}
```

Introducción RTOS

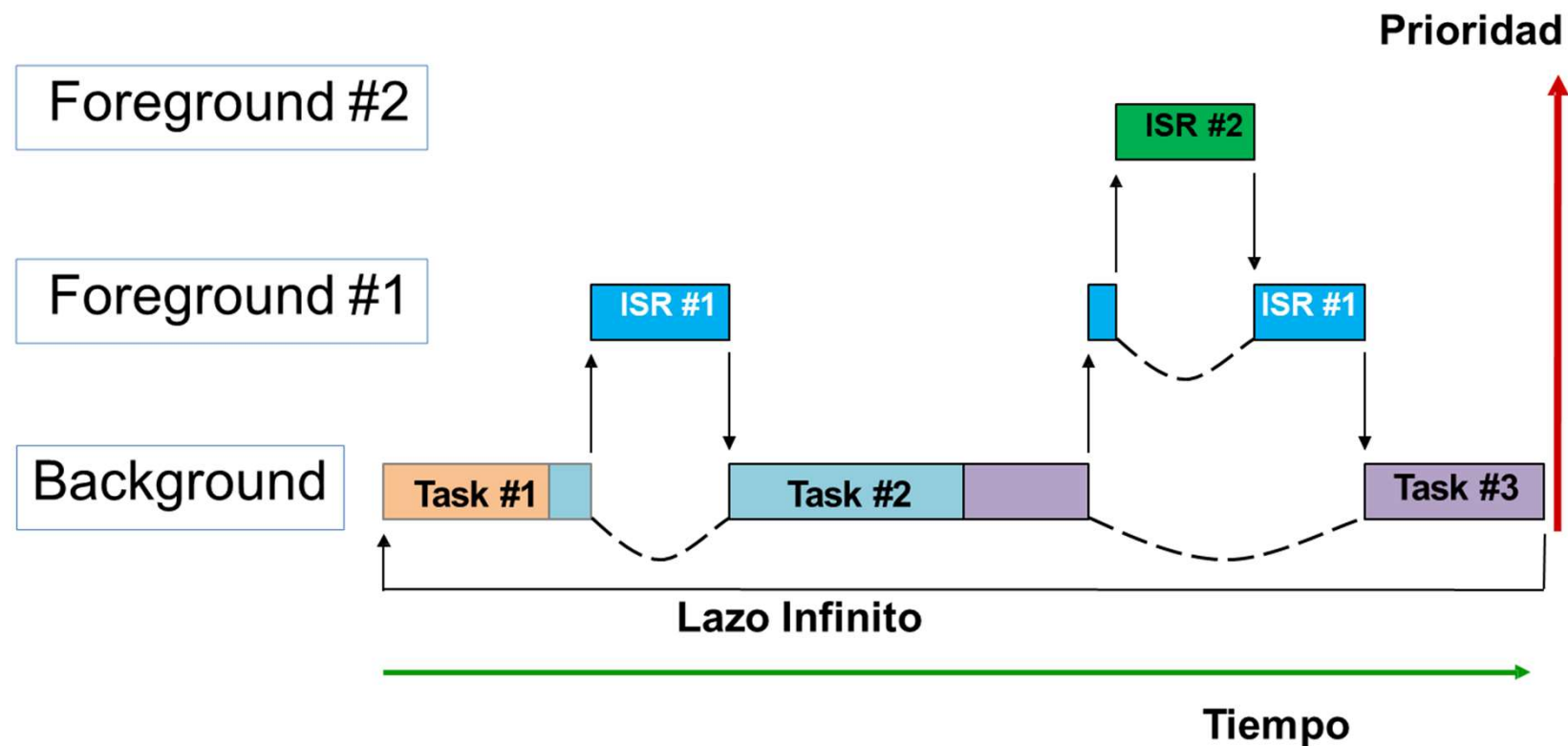
Sistemas Foreground/Background

- Un programa consiste en:
 - **Background** (o nivel de tarea, *task level*): bucle infinito que llama funciones para realizar operaciones no críticas temporalmente (Puede incluir *Watchdogs* de las funciones *foreground*).
 - **Foreground** (o nivel de interrupción, *interrupt level*): conjunto de rutinas de servicio de interrupción que manejan eventos asíncronos.
- Las prioridades asignadas a las distintas fuentes de interrupción determinan su jerarquía y el tiempo que pasa entre que sucede un evento y que éste es atendido (latencia de servicio).
- No existe un administrador de tareas, sino que las tareas interactúan entre sí, a veces en forma compleja y poco estructurada (soluciones “ad-hoc”).
- Es el más usado cuando se programan microprocesadores en embebidos sin grandes pretensiones de crecimiento de código ni estructuración.

Introducción RTOS

Sistemas Foreground/Background

- Puede existir anidamiento de ISR



Introducción RTOS

Sistemas Foreground/Background

■ Ejemplo

```
/* Background */
void main (void)
{
    Initialization;

    FOREVER { // while (1)..for (;;)
        Read analog inputs;
        Read discrete inputs;
        Perform monitoring functions;
        Perform control functions;
        Update analog outputs;
        Update discrete outputs;
        Scan keyboard;
        Handle user interface;
        Update display;
        Handle communication requests;
        Other...
    }
}
```

```
/* Foreground */
ISRa (void) // prioridad media
{
    Enable higher interrupts;
    Handle asynchronous event;
}

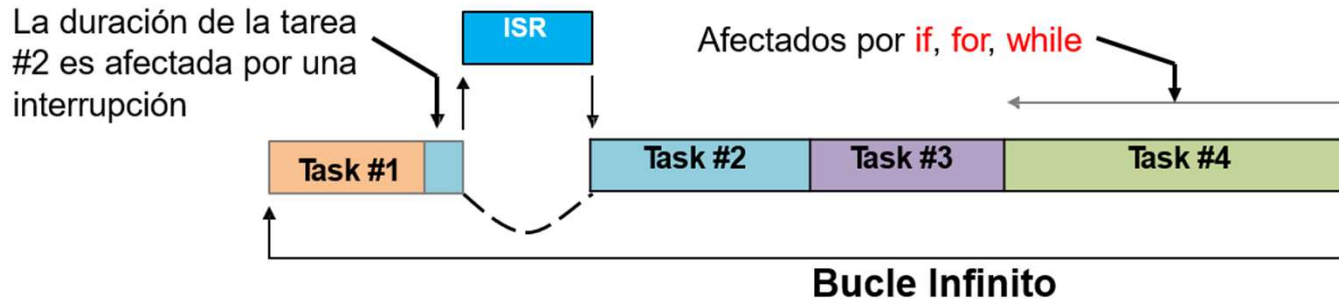
ISRb (void) // prioridad alta
{
    Handle asynchronous event;
}
```

Un bucle infinito que atiende las tareas de soporte, donde las tareas prioritarias son atendidas por interrupciones

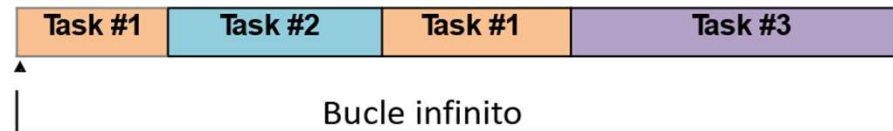
Introducción RTOS

Problemas asociados sin RTOS

- Los sistemas sin administrador de tareas:
 - Son útiles para sistemas sencillos
 - Pero las tareas en *background* (sin *ISR*) tienen poca previsibilidad temporal



- Las tareas en *background* tienen todas la misma prioridad. A menudo se necesita repetir ejecuciones en el bucle para no perder eventos



- Se suelen implementar a medida servicios como : retardos (bucles nops), manejadores de E/S, ... con muy difícil mantenimiento y crecimiento de código

Introducción RTOS

Bibliografía

- Real-Time Systems Design and Analysis.
 - 4th ed. P. Laplante y S. Ovaska. John Wiley and Sons, 2012
- Sistemas Operativos. Diseño e Implementación.
 - 2ª ed. A. S. Tanenbaum y E. Woodhull. Prentice Hall, 1998
- MicroC/OS-II. The Real-Time Kernel.
 - 2nd ed. J. Labrosse. CMP Books, 2002
- Real-Time Embedded Systems. Design Principles and Engineering Practices.
 - Xiaocong Fan. Elsevier 2015.
- Hardware-dependent Software. Principles and Practice
 - Wolfgang Ecker, Wolfgang Müller, Rainer Dömer. Springer 2009.
- Reusable Firmware Development. A Practical Approach to APIs, HALs and Drivers.
 - Jacob Benigno, 2017